

SupplyScore - Vendor & Supplier Scorecard for Product Businesses

A Mini Project Report

submitted by

HUDA (KITM24MCA-2010)

To

The APJ Abdul Kalam Technological University
In partial fulfillment of the requirements for the award of the Degree

of

Master of Computer Applications



Department of Computer Applications

KMCT ITM
Kuttippuram, Malappuram - 679571

OCTOBER 2025

DECLARATION

I undersigned hereby declare that the project report **SUPPLYSCORE**, submitted for partial fulfillment of the requirements for the award of degree of Master of Computer Applications of the APJ Abdul Kalam Technological University, Kerala, is a bona fide work done by me under supervision of Supervisor, Assistant Professor, Department of Computer Applications. This submission represents my ideas in my own words and where ideas or words of others have been included. I have adequately and accurately cited and referenced the original sources. I also declare that I have adhered to ethics of academic honesty and integrity and have not misrepresented or fabricated any data or idea or fact or source in my submission. I understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been obtained. This report has not been previously formed the basis for the award of any degree, diploma or similar title of any other University.

Place:Kuttippuram

Date:17/10/2025

DEPARTMENT OF COMPUTER APPLICATIONS
KMCT ITM, KUTTIPPURAM



CERTIFICATE

This is to certify that the report entitled **SUPPLYSCORE** is a Bonafide record of the Mini Project work during the year 2024-25 carried out by **HUDA(KITM24MCA-2010)** submitted to the APJ Abdul Kalam Technological University, in partial fulfillment of the requirements for the award of the Master of Computer Applications, under my guidance and supervision. This report in any form has not been submitted to any other University or Institution for any purpose.

Internal Supervisor(s)

External Supervisor(s)

Head of The Department

ACKNOWLEDGEMENT

At the very outset I would like to thank the almighty's mercy towards me over the years. I wish to express my sincere thanks to my project coordinator, Ms. LAKSHMI R MENON Head of the department, Dept. of Master of Computer Applications who guided me for the successful completeness of this project. I also thank her for valuable suggestions, guidance, constant encouragement, boundless corporation, constructive comments and motivation extended to me for completion of this project work.

I would express my sincere thanks to my internal guide Ms. LAKSHMI R MENON, guidance to complete the project successfully.

I would like to express my sincere thanks to all the faculty members of Master of Computer Applications department for their support and valuable suggestion for doing the project work. Last but not least my graceful thanks to my parents, friends and also the persons who supported me directly and indirectly during the project.

HUDA (KITM24MCA-2010)

ABSTRACT

In today's competitive business environment, efficient vendor and supplier management has become a cornerstone of success for product-based companies, manufacturers, and wholesale distributors. The quality, reliability, and performance of suppliers directly influence a company's operational efficiency, production timelines, and customer satisfaction. However, traditional methods of vendor evaluation—such as manual scorecards, spreadsheets, and subjective reviews—are time-consuming, error-prone, and lack transparency.

“SupplyScore” is a comprehensive web-based platform developed to automate and streamline the process of vendor and supplier evaluation. The system provides a centralized and data-driven framework to assess suppliers across multiple performance parameters, including cost efficiency, product quality, delivery punctuality, responsiveness, and regulatory compliance. By integrating these metrics into a unified scorecard system, SupplyScore helps businesses make objective, consistent, and evidence-based sourcing decisions.

The platform includes key modules such as vendor registration and authentication, performance tracking, review and rating management, quote comparison, dispute resolution, and interactive analytics dashboards. These modules collectively enhance visibility and accountability across the supply chain while minimizing manual workload. The analytics dashboard visualizes supplier trends, performance statistics, and historical data using charts and reports, supporting strategic decision-making.

Developed using Python (Django framework) for the backend and HTML, CSS, Bootstrap, and JavaScript for the frontend, SupplyScore ensures a responsive and user-friendly interface. The system employs a relational database (MySQL or PostgreSQL) for secure and efficient data management, and JWT-based authentication for secure user access.

Overall, SupplyScore transforms vendor management from a subjective, paperwork-driven activity into a transparent, automated, and performance-oriented process, thereby improving supplier relationships, reducing operational risks, and enhancing long-term supply chain efficiency.

CONTENTS

Declaration	i
Certificate	ii
Acknowledgements	iii
Abstract	iv
Contents	v
List of Figures	vi
1 Introduction	
1.1 Background	7
1.1.1 SUPPLYSCORE	7
1.2 Components	8
1.3 Motivation	9
1.4 Objective	9
1.5 Contribution	10
1.6 Report Organization	10
2 Existing System	11
3 Methodology	12
3.1 Introduction	12
3.2 Workflow	13
3.3 User Story	15
3.4 Product Backlog	16
3.5 Sprint Backlog	17
4 Design	20
4.1 Result	20
4.2 Data Flow Diagram	23
4.3 ER Diagram	25
4.4 Table	25
5 Conclusions	
5.1 Reference	27
6 APPENDIX	
6.1 Source Code	

LIST OF FIGURES

Project Plan	16
Product Backlog	17
Sprint Backlog 1	17
Sprint Backlog 2	17
Sprint Backlog 3	18
Sprint Backlog Actual 1	18
Sprint Backlog Actual 2	18
Sprint Backlog Actual 3	19
Data flow Diagram	20
ER diagram	23
Result	24
Table	25

Chapter1

Introduction

This project titled “SupplyScore – Vendor & Supplier Scorecard for Product Businesses” focuses on building a digital platform for efficient vendor evaluation, performance tracking, and supplier management. With the growing complexity of supply chains and the need for timely, data-driven decisions, businesses require a system that simplifies vendor comparison, automates performance monitoring, and maintains transparency in procurement processes.

SupplyScore is designed to enable businesses to manage suppliers effectively, providing tools to evaluate, rate, and compare vendors based on critical metrics such as cost, quality, delivery speed, and compliance. Businesses can track past transactions, access verified reviews, resolve disputes, and make informed sourcing decisions, while suppliers benefit from structured feedback and performance analytics. By integrating vendor management, quote comparison, dispute resolution, and analytics, the system addresses the challenges of traditional manual processes and fragmented supplier evaluation.

The project is implemented using Python (Django/Flask) as the backend framework and PostgreSQL/MySQL as the database, ensuring a secure, scalable, and user-friendly solution for modern product-based businesses.

1.1 Background

1.1.1 SupplyScore

In recent years, product-based businesses, manufacturers, and wholesale distributors have faced increasing challenges in managing vendors and suppliers efficiently. Traditional vendor management methods, though functional, are often limited by manual processes, lack of standardized evaluation methods, and minimal transparency in performance tracking. Existing systems may focus on either procurement or review management, but few provide an integrated platform for evaluation, comparison, and dispute resolution. SupplyScore bridges this gap by offering a digital solution that combines vendor evaluation, quote comparison, dispute management, and performance analytics. By enabling businesses to rate, review, and track vendors in a structured way, SupplyScore

strengthens supplier relationships, enhances decision-making, and improves overall supply chain efficiency.

1.2 Components

- **Client Interface:**

The client interface interprets inputs from the user and displays the output through a user-friendly web interface. It allows businesses and suppliers to perform interactions such as vendor registration, submitting and reviewing quotes, logging disputes, and viewing analytics dashboards. The interface is dynamic and responsive, ensuring smooth navigation and interaction for all users.

- **Middleware/Backend:**

The backend serves as the core processing unit of the system. Implemented using Python (Django/Flask), it handles requests from the client, executes the business logic, and communicates with the database. This layer ensures secure authentication (using JWT), accurate calculations of vendor scores, and proper management of all modules such as vendor review, quote comparison, and dispute resolution.

- **Database:**

The database stores all critical information including user details, vendor profiles, reviews, quotes, disputes, and purchase histories. Using PostgreSQL/MySQL, and Django's ORM (Object Relational Mapping), SupplyScore securely handles data operations while maintaining consistency and reliability.

- **Platform:**

SupplyScore is implemented as a browser-based application, making it platform-independent and easily accessible without software installation. Users can access the system from any device with a web browser, ensuring flexibility, scalability, and broad usability.

1.3 Motivation

The primary motivation behind SupplyScore is the realization that businesses often struggle with fragmented and manual vendor management processes. Challenges include:

Difficulty in comparing multiple vendor quotes.

Limited visibility into vendor performance and reliability.

Time-consuming dispute resolution processes.

A centralized, digital system that automates evaluation, provides verified reviews, and tracks performance over time can reduce inefficiencies, improve decision-making, and minimize risks. SupplyScore was designed to address these pain points and streamline vendor management effectively.

1.4 Objective

The main objectives of the SupplyScore project are:

- To design a platform where businesses can register, evaluate, and track vendors efficiently.
- To enable secure vendor registration and authentication.
- To allow businesses to submit quotes and compare vendor proposals based on multiple performance metrics.
- To provide a structured system for vendor reviews, ratings, and dispute logging.
- To offer performance analytics and dashboards for informed decision-making.
- To reduce reliance on manual processes through automation and data-driven insights.
- To deliver a secure, user-friendly, and GUI-oriented platform for supplier management.

1.5 Contribution

SupplyScore contributes to the business and supply chain community by providing a comprehensive digital vendor management platform. Unlike traditional systems that manage vendors manually or in a fragmented manner, SupplyScore integrates multiple functionalities into a single platform, including:

- Vendor evaluation and performance tracking.
- Quote comparison and dispute management.
- Verified reviews and ratings to improve transparency.
- Analytics dashboards for monitoring and decision-making.

This approach enhances operational efficiency, reduces risks associated with unreliable

suppliers, and promotes better business relationships. By automating and structuring vendor management, SupplyScore ensures reliability, accountability, and actionable insights for businesses.

1.6 Report Organization

This report is organized as follows:

- Chapter 1: Introduces the project, its background, motivation, objectives, and contributions.
- Chapter 2: Discusses existing vendor management systems and highlights their limitations.
- Chapter 3: Explains the methodology adopted for the development of SupplyScore, including system design practices.
- Chapter 4: Presents the design of the system, including data flow diagrams, ER diagrams, database schema, and module architecture.
- Chapter 5: Provides results, discussions, and conclusion of the project.
- References and Appendix: Include supporting materials, screenshots, and source code.

Chapter 2

Existing System

In most product-based businesses and manufacturing industries, vendor and supplier management is often carried out through manual methods, spreadsheets, emails, or informal communication channels. While these approaches may work for small operations, they become inefficient, error-prone, and difficult to manage as the number of suppliers and transactions increases.

Currently, businesses depend heavily on manual tracking of vendor performance, past transactions, and dispute resolution. Vendor information, quotes, and reviews are scattered across different sources, making it difficult to access reliable data when making procurement decisions. Resource sharing between departments and comparison of multiple vendor quotes is often inconsistent and unstructured.

The drawbacks of the existing system include:

- Heavy dependency on manual communication and record-keeping.
- Lack of a centralized platform for vendor evaluation, quote comparison, and dispute resolution.
- Redundant or outdated vendor information leading to errors in decision-making.
- Limited transparency and accountability in vendor performance.
- Low security and lack of proper access control for vendor data.
- Difficulty in tracking disputes, performance metrics, and historical transactions.
- Inefficient handling of data analytics for procurement and supplier management.

These challenges highlight the need for a computerized, automated, and integrated system that can evaluate vendors, manage quotes, track performance, resolve disputes, and provide analytics for informed decision-making. SupplyScore has been designed to address these issues by providing a secure, user-friendly, and efficient platform for supplier management.

Chapter 3

Methodology

3.1 Introduction

After initial analysis, it was determined that the Agile methodology is most suitable for the development of SupplyScore. Agile focuses on client satisfaction through continuous delivery, iterative development, and rapid improvements. Since SupplyScore involves multiple functional modules such as user management, vendor review, quote comparison, dispute resolution, and analytics, each module can be developed incrementally. This enables businesses to gradually adopt features and benefit from the system immediately, while allowing improvements based on feedback. Among the Agile approaches, Scrum has been selected for managing development. Scrum is a lightweight framework for complex product development, providing flexibility while ensuring continuous improvement and faster delivery. It encourages collaboration, transparency, and accountability throughout the project lifecycle.

The Scrum framework includes three key roles:

1. Product Owner (PO): Represents stakeholders and businesses, responsible for defining and prioritizing requirements.
2. Scrum Master: Ensures Scrum processes are followed, removes obstacles, and supports the development team.
3. Development Team: A cross-functional, self-organizing group responsible for design, implementation, testing, and delivery.

Work is carried out in sprints (fixed-duration iterations). The Product Owner maintains a Product Backlog, from which the team selects high-priority items to complete during each sprint. Scrum can be summarized using the 3-3-5 framework:

- 3 Roles: Product Owner, Scrum Master, Development Team

- 3 Artifacts: Product Backlog, Sprint Backlog, Product Increment
- 5 Events: Sprint, Sprint Planning, Daily Scrum, Sprint Review, Sprint Retrospective

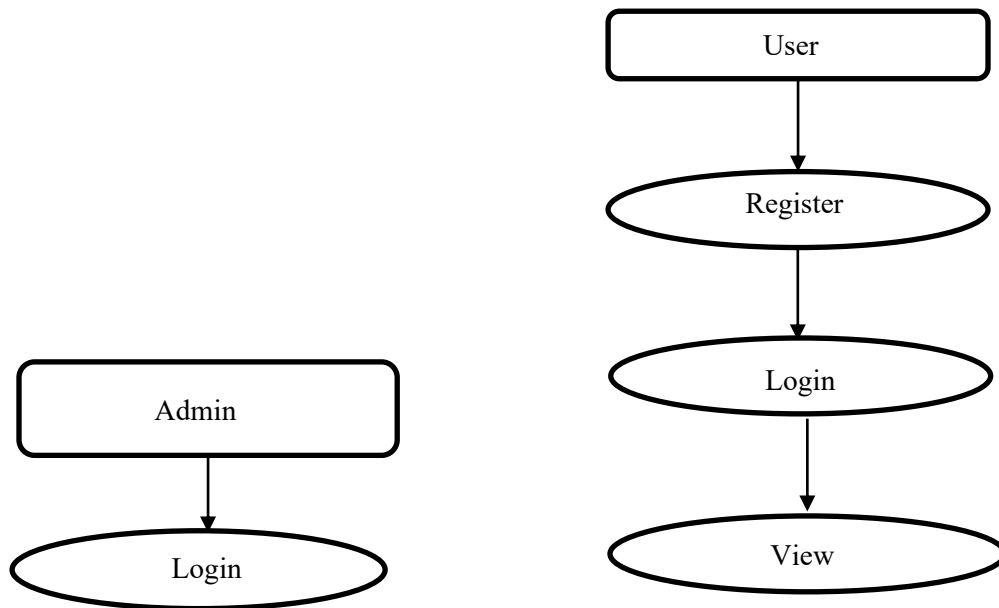
The three pillars of Scrum — Transparency, Inspection, and Adaptation — ensure that SupplyScore is developed efficiently with maximum collaboration and accountability. For the development of SupplyScore, Python Django (or Flask) is used for the backend. Django provides a secure, scalable environment for handling multiple user roles such as business users and vendors. It supports features like user authentication, vendor evaluation, quote comparison, dispute management, and analytics dashboards. The project uses PostgreSQL/MySQL as the database for reliable data storage and easy integration with Django's ORM.

3.2.WORKFLOW

The workflow of SupplyScore development follows Agile principles and modern development tools:

- Development Environment: Visual Studio Code is used as the primary code editor for its flexibility, cross-platform support, and integration with version control systems.
- Version Control: Git is used to track and manage code changes, enabling collaboration, rollback, and efficient project management.
- Backend Framework: Django (or Flask) is used for server-side development to manage user authentication, vendor and business modules, and analytics.
- Frontend & Styling: HTML, CSS, JavaScript, and Bootstrap/Tailwind CSS are used for a responsive and intuitive user interface.
- Database: PostgreSQL or MySQL is used to store user information, vendor data, reviews, quotes, disputes, and performance analytics.
- Core Workflow:
 - Businesses can register, submit quotes, rate vendors, track vendor performance, and log disputes.
 - Vendors can register, respond to quotes, receive ratings, and view performance analytics.
 - Admins can manage users, oversee disputes, and monitor the entire system.

SupplyScore ensures smooth collaboration between businesses and vendors, efficient evaluation, and structured analytics. Iterative development with Agile allows gradual feature release, continuous improvement, and reliable adoption of the system in real-world business scenarios.



3.3. USER STORY

USER STORY ID	AS A	I WANT TO	SO THAT I CAN
1	Guest User	browse available vendors and supplier performance scores without an account	get an idea of the platform features and vendor options before signing up
2	Guest User	create a new account	register as a business or vendor to access full platform features
3	Business User	log into my account securely	access my dashboard, vendor management tools, and quote comparison features
4	Business User	view and compare vendor quotes for products/services	make informed procurement decisions efficiently

5	Business User	rate and review vendors after a transaction	provide feedback and maintain an accurate vendor rating system
6	Business User	track vendor performance over time	monitor reliability, quality, and delivery efficiency for better sourcing decisions
7	Business User	raise a dispute with a vendor	resolve issues related to products, services, or delivery problems
8	Business User	view my transaction and dispute history	keep track of past orders, reviews, and dispute resolutions
9	Business User	save favorite vendors or create a preferred vendor list	quickly access reliable vendors for future transactions
10	Vendor	register as a vendor and create my profile	showcase my products/services and participate in quote submissions
11	Vendor	respond to business requests for quotes	provide competitive pricing and secure orders
12	Vendor	view ratings and reviews given by businesses	improve service quality and maintain credibility
13	Vendor	track my performance metrics on the dashboard	monitor delivery, compliance, and quality over time
14	Vendor	manage disputes raised by businesses	resolve conflicts and maintain strong client relationships
15	Admin	view and manage all user accounts	ensure platform safety and suspend problematic accounts
16	Admin	review and remove inappropriate vendor listings or reviews	maintain quality and reliability of the platform
17	Admin	view platform statistics (active businesses, vendors, disputes, ratings)	analyze platform usage and make data-driven decisions

18	Admin	broadcast announcements to all users	inform users about policy updates, new features, or maintenance
----	-------	--------------------------------------	---

3.4. PRODUCT BACKLOG

ID	PRIORITY	SIZE(HOURS)	SPRINT	STATUS	NAME
1	medium	4	1	planned	Registration
2	medium	4		planned	Login
3	High	10	2	Partially completely	Table Design
4	High	22	3	Planned	Coding
5	Medium	10	4	Planned	Testing Data
6	High	5	5	planned	Output generation

3.5. PROJECT PLAN

USER STORY ID	SPRINT	START DATE	END DATE	HOURS	STATUS	NAME
US-1 to US-6	Sprint 1	01/07/2025	18/07/2025	10	Complete	Registration, Login
US-7 to US-13	Sprint 2	23/07/2025	30/07/2025	25	Complete	Table Design, Coding
US-14 to US-19	Sprint 3	01/08/2025	24/10/2025	15	Complete	Testing Data, Output Generation

3.6. SPRINT BACKLOG

SPRINT 1:

Backlog Item	Status & Completion Date	Original Estimate (Hours)	Day 1 01/07	Day 2 02/07	Day 3 04/07	Day 4 08/07	Day 5 11/07	Day 6 15/07	Day 7 16/07	Day 8 18/07
User Registration Module	05/07/25	3	1	1	1	0	0	0	0	0
User Login & Authentication	06/07/25	3	1	0	1	1	0	0	0	0

Vendor Profile Management	08/07/25	4	0	1	1	1	1	0	0	0
Total		10	2	2	3	2	1	0	0	0

SPRINT 2:

Backlog Item	Status & Completion Date	Original Estimate (Hours)	Day 1 23/07	Day 2 25/07	Day 3 27/07	Day 4 28/07	Day 5 29/07	Day 6 30/07
Quote Management (Request & Compare)	25/07/25	6	1	1	1	1	1	1
Product & Vendor Listing Page	27/07/25	6	1	1	1	1	1	1
Scorecard Evaluation Logic	28/07/25	6	1	1	1	1	1	1
Dashboard UI Design	29/07/25	6	1	1	1	1	1	1
Database Integration & Testing	30/07/25	6	2	2	1	1	0	0
Total		30	6	6	5	5	4	4

SPRINT 3:

Backlog Item	Status & Completion Date	Original Estimate (Hours)	Day 1 01/08	Day 2 05/08	Day 3 10/08	Day 4 15/08	Day 5 20/08	Day 6 24/08	Day 7 29/08	Day 8 24/10
Vendor Review & Rating System	05/08/25	4	1	1	1	1	0	0	0	0
Dispute Management Module	10/08/25	5	1	1	1	1	1	0	0	0

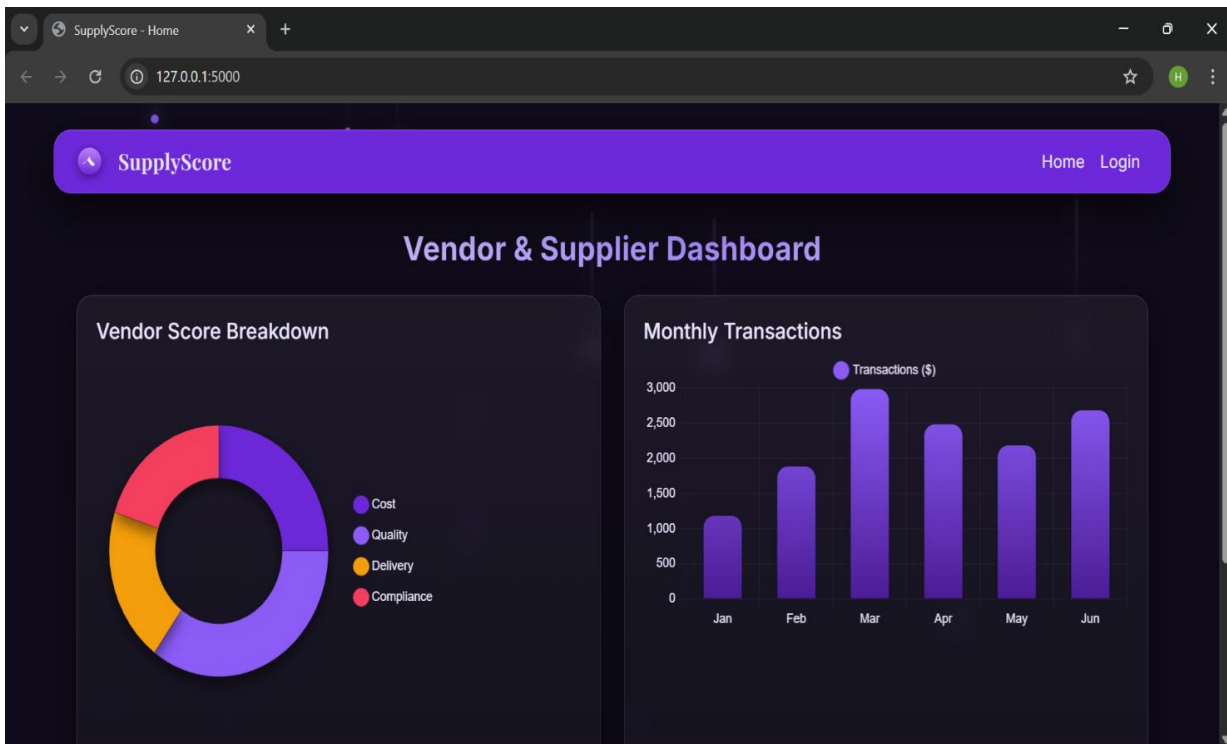
Purchase & Transaction History	15/08/25	4	1	1	1	1	0	0	0	0
Notifications & Alerts	20/08/25	4	1	1	1	1	0	0	0	0
Admin Panel Features	24/10/25	5	0	0	1	1	1	1	1	0
Total		22	4	4	5	5	2	1	1	0

Chapter 4

Design

4.1 RESULT

Home Page



Register Page

Register - SupplyScore

127.0.0.1:5000/register

Register

Name:

Enter your name

Email:

Enter your email

Password:

Enter your password

Role:

Business

Register

Login Page

Login - SupplyScore

127.0.0.1:5000/login

Login

Email:

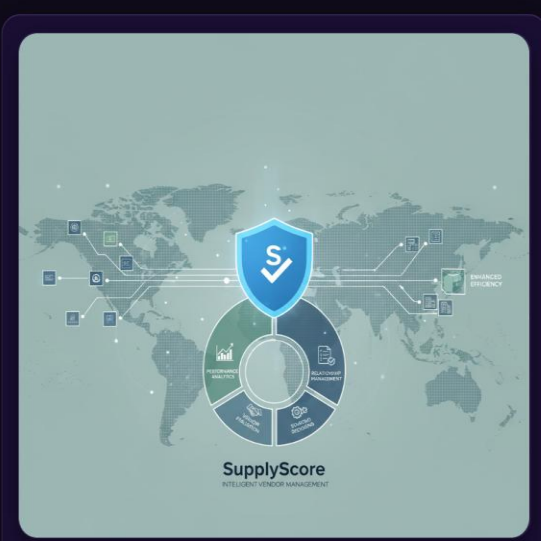
Enter your email

Password:

Enter your password

Login

New here? [Create an account](#)



The graphic on the right side of the login page features a world map with various icons representing different aspects of supply chain management, such as a shield with a checkmark, a gear, a document, and a bar chart. The text 'SupplyScore' is prominently displayed in the center, with 'INTELLIGENT VENDOR MANAGEMENT' written below it.

Vendor Review Page

Vendor Review - SupplyScore

127.0.0.1:5000/vendor-review

Vendor Review

[Home](#)

Logged in successfully.

Your Reviews

Review #3 2025-09-27 15:49

Vendor ID: 6 **Quality:** ★★★★★

Cost-effectiveness: ★★★★★ **Delivery Speed:** ★★★★☆

Compliance: ★★★★★ **Overall:** ★★★★☆

Comments:
Good service altogether.

Delete Review

Review #2 2025-09-27 15:48

Vendor ID: 4 **Quality:** ★★★☆☆

Cost-effectiveness: ★★★★★ **Delivery Speed:** ★★★★★

Compliance: ★★★★★ **Overall:** ★★★★☆

Comments:
Need to have better price range and quality improvement.

Delete Review

4.2 Data Flow Diagram

i. Level 0

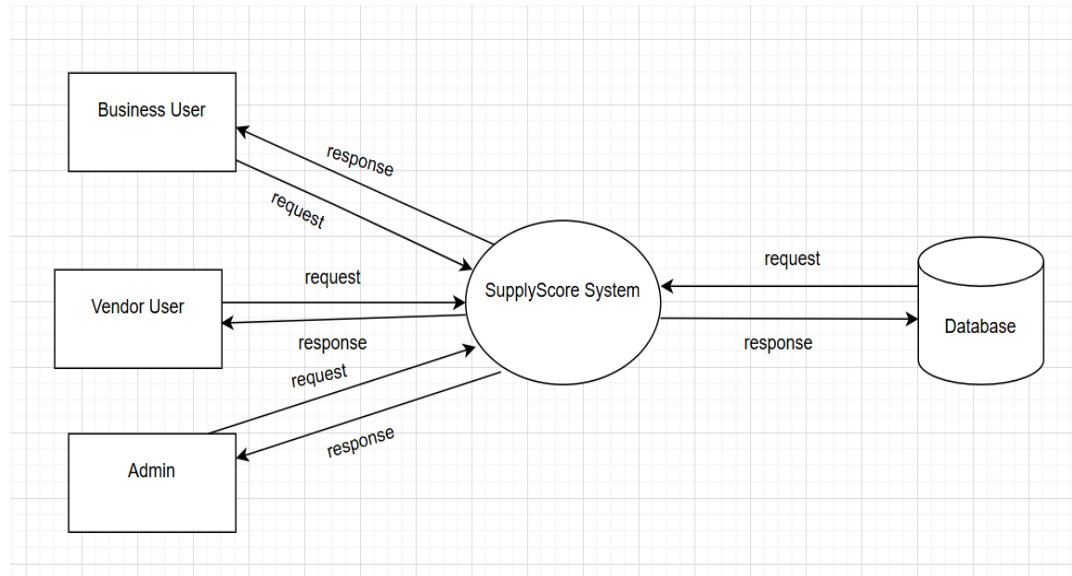


Figure A.1: Dataflow Diagram

ii. Level 1

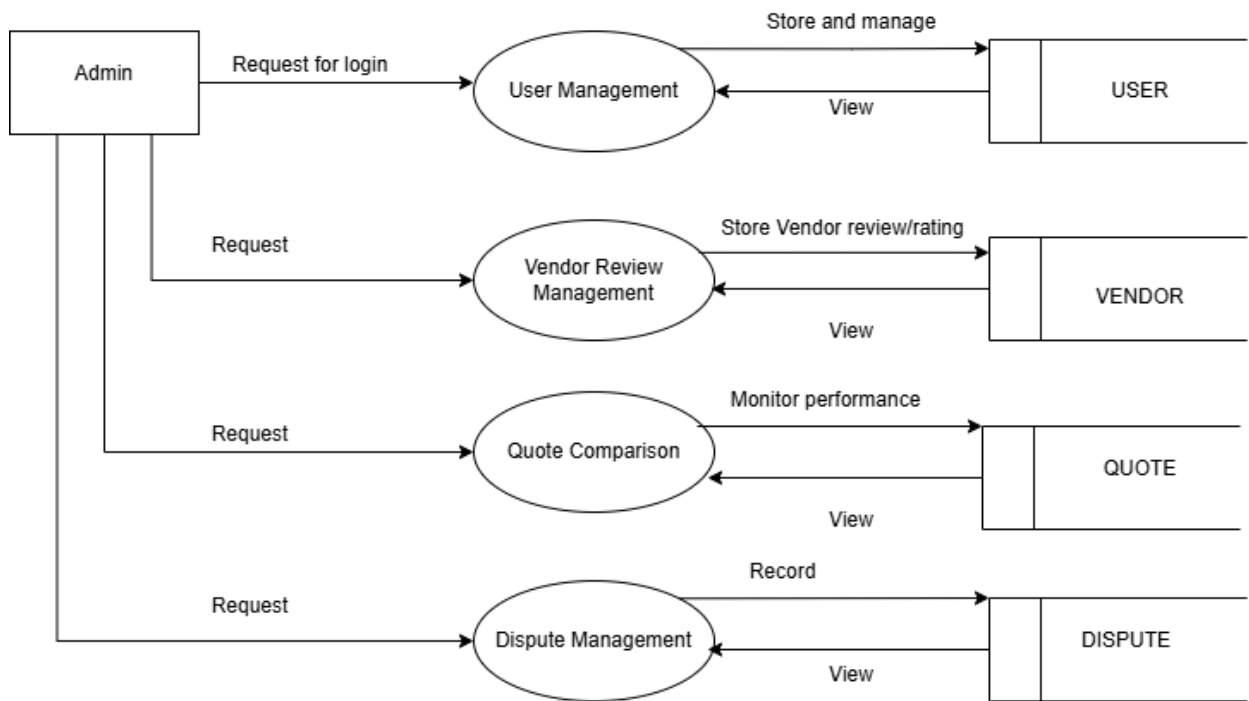


Figure A.2 Dataflow Diagram

iii. Level 2

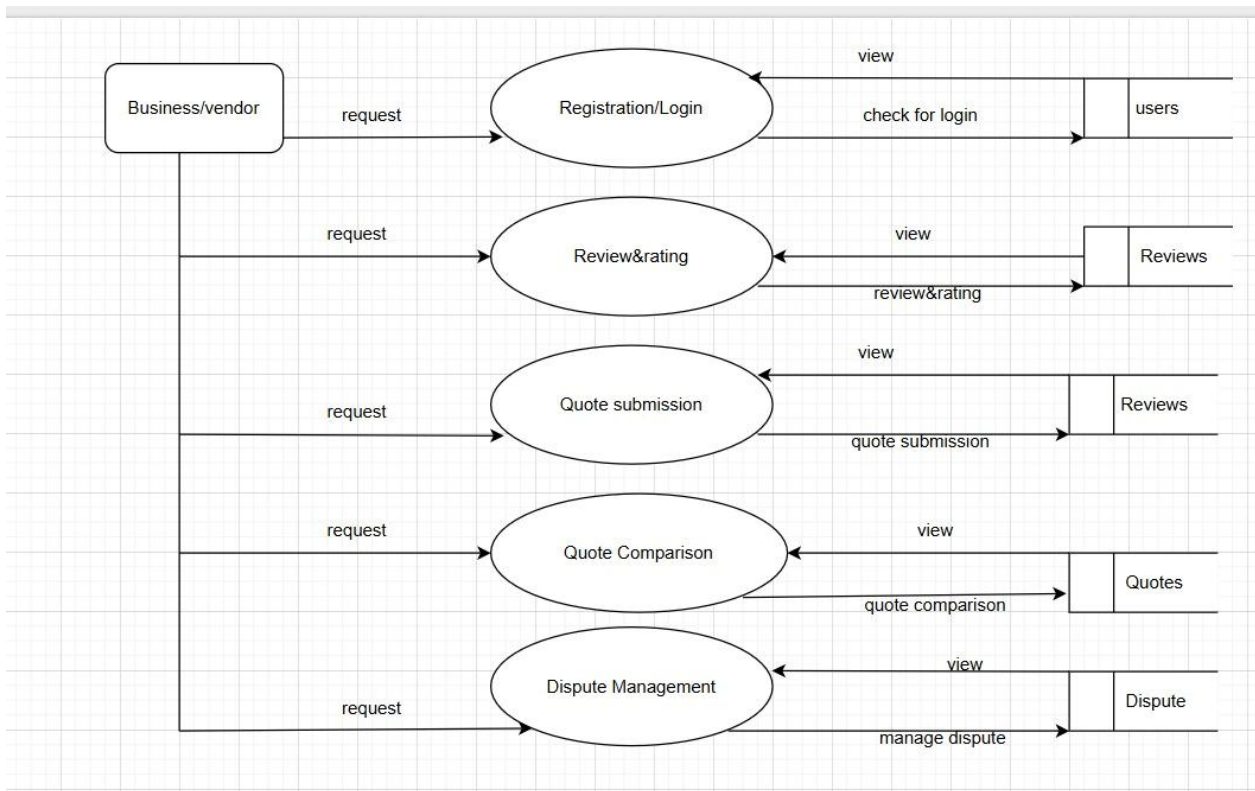
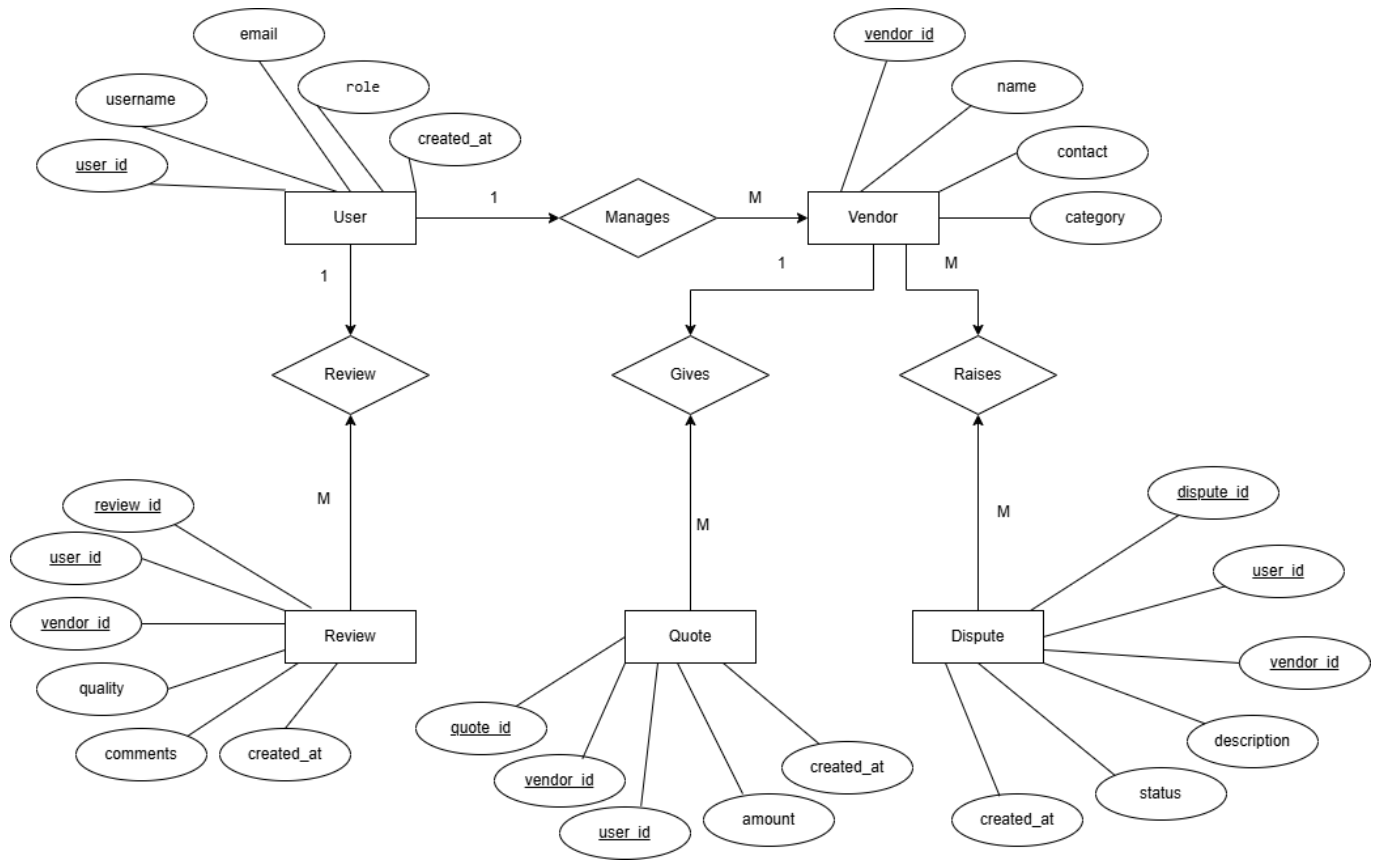


Figure A.3: Dataflow Diagram

4.3 ER DIAGRAM



4.4 TABLE

USER

Field Name	Data Type	Constraints
user_id	Integer	Primary Key
name	Varchar(100)	Not Null
email	Varchar(100)	Unique, Not Null
password	Varchar(100)	Not Null

VENDOR

Field Name	Data Type	Constraints
vendor_id	Integer	Primary Key
vendor_name	Varchar(100)	Not Null
contact_info	Varchar(100)	Not Null
performance_score	Float	Default 0

REVIEW

Field Name	Data Type	Constraints
review_id	Integer	Primary Key
user_id	Integer	Foreign Key (references USER.user_id)
vendor_id	Integer	Foreign Key (references VENDOR.vendor_id)
cost_rating	Integer	Not Null
quality_rating	Integer	Not Null

QUOTE_COMPARISON

Field Name	Data Type	Constraints
quote_id	Integer	Primary Key
vendor_id	Integer	Foreign Key (references VENDOR.vendor_id)
product_name	Varchar(100)	Not Null
quoted_price	Float	Not Null
delivery_time	Varchar(50)	Not Null
comparison_date	comparison_date	comparison_date

5 CONCLUSION

The project “SupplyScore – Vendor & Supplier Scorecard for Product Businesses” successfully provides a digital platform for managing vendor evaluations, supplier comparisons, and dispute handling in a structured and transparent manner. The system automates what was previously a manual, time-consuming, and error-prone process by integrating all key functions—review management, quote comparison, performance analysis, and dispute resolution—into a single web-based interface.

The application, developed using Python Django and MySQL, ensures secure authentication, accurate data handling, and user-friendly navigation for businesses, vendors, and administrators. Since the system is browser-based, it requires no local installation, which makes deployment, access, and maintenance much easier for multiple users.

Compared to traditional spreadsheet-based evaluations or offline methods, SupplyScore significantly improves efficiency by generating real-time insights through vendor performance metrics and dashboards. The use of Django templates and Bootstrap enhances both speed and responsiveness, while the modular structure allows for easy expansion of future features such as analytics visualization, automated alerts, and AI-based performance prediction.

Furthermore, the application reduces redundancy, improves decision-making accuracy, and promotes fairness in supplier management. The system’s ability to calculate, store, and visualize performance data offers an intelligent and data-driven approach to vendor evaluation.

Overall, the successful implementation of SupplyScore demonstrates how web-based automation can streamline procurement and vendor management processes, helping organizations strengthen their supply chains and achieve better operational outcomes. Future enhancements may include integrating APIs for live data import, predictive scoring, and cloud-based deployment for enterprise scalability.

REFERENCES

- [1] Chopra, S., & Meindl, P. (2019). *Supply Chain Management: Strategy, Planning, and Operation* (7th Edition). Pearson Education.
- [2] Monczka, R. M., Handfield, R. B., Giunipero, L. C., & Patterson, J. L. (2020). *Purchasing and Supply Chain Management* (7th Edition). Cengage Learning.
- [3] Heizer, J., Render, B., & Munson, C. (2017). *Operations Management: Sustainability and Supply Chain Management* (12th Edition). Pearson.
- [4] ISO 9001:2015 – Quality Management Systems. *International Organization for Standardization (ISO)*. Available at: <https://www.iso.org>
- [5] W3Schools. (2024). *HTML, CSS, JavaScript, Django Tutorials*. Retrieved from <https://www.w3schools.com>
- [6] Django Software Foundation. (2024). *Django Documentation – The Web Framework for Perfectionists with Deadlines*. Retrieved from <https://docs.djangoproject.com>
- [7] MySQL Documentation. (2024). *MySQL 8.0 Reference Manual*. Oracle Corporation. Retrieved from <https://dev.mysql.com/doc>
- [8] Chart.js Developers. (2024). *Chart.js Documentation*. Retrieved from <https://www.chartjs.org/docs/latest/>
- [9] Sahu, R., & Singh, S. (2021). “Vendor Evaluation System for Supply Chain Optimization.” *International Journal of Advanced Research in Computer Science and Software Engineering*, Vol. 11(5), pp. 123–129.
- [10] Kumar, A., & Sharma, V. (2022). “Automated Supplier Performance Evaluation Using Web Technologies.” *International Journal of Emerging Trends in Engineering and Development*, Vol. 9(4), pp. 45–52.
- [11] OpenAI. (2024). *ChatGPT for Project Assistance and Documentation*. Retrieved from <https://chat.openai.com>
- [12] MDN Web Docs. (2024). *JavaScript, HTML, CSS, and Web API Documentation*. Retrieved from <https://developer.mozilla.org>

APPENDIX

Source Code

home.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>SupplyScore - Home</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
  <link href="{{ url_for('static', filename='core/css/style.css') }}" rel="stylesheet">

  <!-- Chart.js -->
  <script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.4/dist/chart.umd.min.js"></script>
</head>
<body>

<div class="bg-particles" aria-hidden="true">
  <span></span><span></span><span></span><span></span><span></span>
  <span></span><span></span><span></span><span></span><span></span>
  <span></span><span></span><span></span>
</div>

<!-- Navbar -->
<nav class="navbar navbar-expand-lg navbar-dark bg-primary">
  <div class="container">
    <a class="navbar-brand" href="{{ url_for('core.home') }}">
      <svg class="brand-icon" viewBox="0 0 32 32" xmlns="http://www.w3.org/2000/svg" aria-
hidden="true">
        <defs>
          <linearGradient id="g1" x1="0" x2="0" y1="0" y2="1">
            <stop offset="0%" stop-color="#c084fc"/>
            <stop offset="100%" stop-color="#6d28d9"/>
          </linearGradient>
        </defs>
        <circle cx="16" cy="16" r="14" fill="url(#g1)" />
        <path d="M9.5 17.5c2.8-1.2 4.9-3.7 6.1-6.6 1.3 3.4 3.8 6.3 7 8.1c-2.3 2.3c-2.9-1.5-5.2-3.7-6.8-6.5-1.2
1.6-2.8 2.9-4.6 3.8l-1.4-1.1z" fill="#fff" opacity="0.9"/>
      </svg>
      <span class="brand-mark">SupplyScore</span>
    </a>
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarNav">
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="navbarNav">
```

```

        <ul class="navbar-nav ms-auto">
        <li class="nav-item"><a class="nav-link" href="{{ url_for('core.home') }}">Home</a></li>
        {% if session.get('user_role') == 'business' %}
        <li class="nav-item"><a class="nav-link" href="{{ url_for('core.vendor_review') }}">Vendor
Review</a></li>
        <li class="nav-item"><a class="nav-link" href="{{ url_for('core.quote_comparison') }}">Quote
Comparison</a></li>
        {% endif %}
        {% if session.get('user_role') == 'vendor' %}
        <li class="nav-item"><a class="nav-link" href="{{ url_for('core.dispute_management') }}">Dispute
Management</a></li>
        {% endif %}
        {% if session.get('user_role') == 'admin' %}
        <li class="nav-item"><a class="nav-link" href="{{ url_for('core.users') }}">Users</a></li>
        {% endif %}
        {% if session.get('user_id') %}
        <li class="nav-item"><a class="nav-link" href="{{ url_for('core.logout') }}">Logout</a></li>
        {% else %}
        <li class="nav-item"><a class="nav-link" href="{{ url_for('core.login') }}">Login</a></li>
        {% endif %}
        </ul>

    </div>
</div>
</nav>

<!-- Dashboard Content -->
<div class="container">
    <h2 class="mb-4 text-center wave-heading">Vendor & Supplier Dashboard</h2>
    <div class="row">
        <div class="col-md-6">
            <div class="dashboard-card">
                <h5>Vendor Score Breakdown</h5>
                <div class="chart-container">
                    <canvas id="scoreChart"></canvas>
                </div>
            </div>
        </div>
        <div class="col-md-6">
            <div class="dashboard-card">
                <h5>Monthly Transactions</h5>
                <div class="chart-container">
                    <canvas id="transactionChart"></canvas>
                </div>
            </div>
        </div>
    </div>
</div>

```



```

<div class="row mt-4">
  <div class="col-md-4">
    <div class="dashboard-card text-center">
      <h6>Total Vendors</h6>
      <p class="display-6 countup" data-target="25">0</p>
    </div>
  </div>
  <div class="col-md-4">
    <div class="dashboard-card text-center">
      <h6>Total Reviews</h6>
      <p class="display-6 countup" data-target="120">0</p>
    </div>
  </div>
  <div class="col-md-4">
    <div class="dashboard-card text-center">
      <h6>Disputes Resolved</h6>
      <p class="display-6 countup" data-target="15">0</p>
    </div>
  </div>
</div>

<!-- Bootstrap JS -->
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>

<!-- Chart.js Scripts -->
<script>
  // Custom shadow plugin to create a pseudo-3D depth
  const shadowPlugin = {
    id: 'customShadow',
    beforeDatasetsDraw(chart, args, opts) {
      const {ctx} = chart;
      ctx.save();
      ctx.shadowColor = opts.shadowColor || 'rgba(0,0,0,0.45)';
      ctx.shadowBlur = opts.shadowBlur || 18;
      ctx.shadowOffsetX = opts.shadowOffsetX || 0;
      ctx.shadowOffsetY = opts.shadowOffsetY || 10;
    },
    afterDatasetsDraw(chart, args, opts) {
      chart.ctx.restore();
    }
  };

  // Utility to build vertical gradient fill
  function verticalGradient(ctx, area, from, to) {
    const g = ctx.createLinearGradient(0, area.top, 0, area.bottom);
    g.addColorStop(0, from);
    g.addColorStop(1, to);
  }

```

```

    return g;
}

// Colors aligned with the dark purple theme
const purple = '#6d28d9';
const purpleDark = '#4c1d95';
const teal = '#22d3ee';
const amber = '#f59e0b';
const rose = '#f43f5e';

// Score doughnut (with inner cutout and shadow to simulate depth)
const scoreCanvas = document.getElementById('scoreChart');
const scoreCtx = scoreCanvas.getContext('2d');
const scoreChart = new Chart(scoreCtx, {
  type: 'doughnut',
  data: {
    labels: ['Cost', 'Quality', 'Delivery', 'Compliance'],
    datasets: [{
      label: 'Vendor Scores',
      data: [25, 35, 20, 20],
      backgroundColor: [purple, '#8b5cf6', amber, rose],
      borderColor: 'rgba(0,0,0,0.15)',
      borderWidth: 1,
      hoverOffset: 8,
    }]
  },
  options: {
    plugins: {
      legend: {
        position: 'right',
        labels: {
          color: '#eae6ff',
          usePointStyle: true,
          pointStyle: 'roundedRect',
          boxWidth: 10,
          padding: 16
        }
      },
      tooltip: {
        backgroundColor: 'rgba(30, 22, 63, 0.9)',
        borderColor: 'rgba(192,132,252,0.25)',
        borderWidth: 1,
        titleColor: '#eae6ff',
        bodyColor: '#eae6ff'
      }
    },
    cutout: '58%',
    radius: '90%'
  }
});

```

```

    },
    plugins: [shadowPlugin]
  });

// Transactions bar with rounded corners, gradient fill, and shadow
const transactionCanvas = document.getElementById('transactionChart');
const transactionCtx = transactionCanvas.getContext('2d');
const transactionChart = new Chart(transactionCtx, {
  type: 'bar',
  data: {
    labels: ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun'],
    datasets: [{
      label: 'Transactions ($)',
      data: [1200, 1900, 3000, 2500, 2200, 2700],
      backgroundColor: (context) => {
        const {chart} = context;
        const {ctx, chartArea} = chart;
        if (!chartArea) { return purple; }
        return verticalGradient(ctx, chartArea, '#8b5cf6', purpleDark);
      },
      borderColor: 'rgba(0,0,0,0.15)',
      borderWidth: 1,
      borderRadius: 12,
      maxBarThickness: 42,
    }]
  },
  options: {
    plugins: {
      legend: {
        labels: {
          color: '#eae6ff',
          usePointStyle: true,
          pointStyle: 'roundedRect',
        }
      },
      tooltip: {
        backgroundColor: 'rgba(30, 22, 63, 0.9)',
        borderColor: 'rgba(192,132,252,0.25)',
        borderWidth: 1,
        titleColor: '#eae6ff',
        bodyColor: '#eae6ff'
      }
    },
    scales: {
      x: {
        ticks: { color: '#eae6ff' },
        grid: { color: 'rgba(192,132,252,0.08)' }
      },

```

```

        y: {
            beginAtZero: true,
            ticks: { color: '#eae6ff' },
            grid: { color: 'rgba(192,132,252,0.08)' }
        }
    },
    plugins: [shadowPlugin]
});

// Count-up animation for KPI cards
function animateCount(el, duration = 1200) {
    const target = parseInt(el.getAttribute('data-target') || '0', 10);
    const start = 0;
    const startTime = performance.now();
    const easeOutCubic = (t) => 1 - Math.pow(1 - t, 3);

    function frame(now) {
        const elapsed = now - startTime;
        const progress = Math.min(1, elapsed / duration);
        const eased = easeOutCubic(progress);
        const current = Math.floor(start + (target - start) * eased);
        el.textContent = current.toLocaleString();
        if (progress < 1) requestAnimationFrame(frame);
    }
    requestAnimationFrame(frame);
}

document.querySelectorAll('.countup').forEach((el, idx) => {
    // Stagger the animations slightly for a nicer effect
    setTimeout(() => animateCount(el, 1200 + idx * 150), idx * 100);
});
</script>

</body>
</html>

```

login.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>Login - SupplyScore</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
  <link rel="stylesheet" href="{{ url_for('static', filename='core/css/style.css') }}">
</head>
<body>
  <div class="container mt-5">
    <div class="row g-4 align-items-center">
      <div class="col-md-6">
        <h1 class="mb-3">Login</h1>
        {% with messages = get_flashed_messages(with_categories=true) %}
        {% if messages %}
          <div class="mt-3">
            {% for category, message in messages %}
              <div class="alert alert-{{ category }}" role="alert">{{ message }}</div>
            {% endfor %}
          </div>
        {% endif %}
        {% endwith %}
        <form method="POST">

          <div class="mb-3">
            <label>Email:</label>
            <input type="email" name="email" class="form-control" placeholder="Enter your email"
required>
          </div>
          <div class="mb-3">
            <label>Password:</label>
            <div class="input-group">
              <input type="password" id="loginPassword" name="password" class="form-control"
placeholder="Enter your password" required>
              <button class="btn btn-outline-light" type="button" id="toggleLoginPassword" aria-
label="Show password">
                <span class="bi bi-eye"></span>
              </button>
            </div>
          </div>
          <button type="submit" class="btn btn-primary">Login</button>
        </form>
        <div class="mt-3">
          <span>New here? </span>
          <a class="link-light" href="{{ url_for('core.register') }}">Create an account</a>
        </div>
      </div>
    </div>
  </div>
```

```

</div>
<div class="col-md-6">
  <div class="dashboard-card glass-card text-center p-3">
    <div class="media-frame mb-3">
      
    </div>
    <div class="brand-lockup d-flex align-items-center justify-content-center gap-2">
      <svg class="brand-icon" viewBox="0 0 32 32" xmlns="http://www.w3.org/2000/svg" aria-
hidden="true">
        <defs>
          <linearGradient id="g2" x1="0" x2="0" y1="0" y2="1">
            <stop offset="0%" stop-color="#c084fc"/>
            <stop offset="100%" stop-color="#6d28d9"/>
          </linearGradient>
        </defs>
        <circle cx="16" cy="16" r="14" fill="url(#g2)" />
        <path d="M9.5 17.5c2.8-1.2 4.9-3.7 6.1-6.6 1.3 3.4 3.8 6.3 7 8.1-2.3 2.3c-2.9-1.5-5.2-3.7-
6.8-6.5-1.2 1.6-2.8 2.9-4.6 3.8-1.4-1.1z" fill="#fff" opacity="0.9"/>
      </svg>
      <span class="brand-mark">SupplyScore</span>
    </div>
  </div>
</div>
</div>
</div>
</div>

<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>
<script>
  (function(){
    const toggle = document.getElementById('toggleLoginPassword');
    const field = document.getElementById('loginPassword');
    if (toggle && field) {
      toggle.addEventListener('click', () => {
        const isPwd = field.getAttribute('type') === 'password';
        field.setAttribute('type', isPwd ? 'text' : 'password');
        toggle.setAttribute('aria-label', isPwd ? 'Hide password' : 'Show password');
      });
    }
  })();
</script>
</body>
</html>

```